Assignment 3

ADVANCED SIMULATION

EPA133A

Authors:

Niels van den Dool (5026717)
Eli Kapteijn (6587380)
Nick Schreurs (6581951)
Lieke Wahlen (5179564)
Mirte Wildeboer (5326265)

March 20, 2026

TU Delft
Delft University of Technology

Contents

1 Data Preparation **2**
- 1.1 Data Preparation Process . . . 2
- 1.2 Assumptions and Processing Steps . . . 2
- 1.3 Special Cases and Adjustments . . . 2
  - 1.3.1 Sourcesink of N104 . . . 2
  - 1.3.2 Disconnection N106 . . . 3
- 1.4 Data to Model translation . . . 4

2 Model Design & Routing Mechanisms **5**
- 2.1 Model adjustments . . . 5
  - 2.1.1 Generating a NetworkX graph . . . 5
- 2.2 Routing and path allocation . . . 6
- 2.3 Integration between MESA and NetworkX . . . 8

3 Experiments & Scenarios **9**

4 Results & Discussion **10**
- 4.1 Network composition and bridge distribution . . . 10
- 4.2 Results of the Scenario . . . 10
- 4.3 Discussion: differences between scenario outcomes . . . 11
- 4.4 Key relationships between KPIs . . . 11

5 Limitations and possible improvements of the model **12**

References **13**

Acknowledgements **14**

1 Data Preparation

1.1 Data Preparation Process

The raw data from the `_roads3.tsv` file was processed to construct a road network for the simulation. The focus is on the main roads N1 and N2. Based on verification with the course staff, it is assumed that all roads starting with N100 up until N299 are side roads connected to the main roads. Therefore, these roads were included in preparing the data.

For N1 and N2, the minimum and maximum chainage values were used to determine total road lengths. Only roads longer than 25 km were included. For these roads, attributes were added, and start and end points were labelled as sourcesink components.

Distances between coordinates were calculated using haversine to ensure geographic distance measurements.

1.2 Assumptions and Processing Steps

Intersections were identified by comparing points from different roads. An intersection is assumed to exist when the distance between two points is smaller than 0.05 ($\approx$ 50 meters). This threshold is an assumption and directly influences the number of detected intersections.

Detected intersection points were grouped to represent a single physical intersection. This ensures that intersections shared by multiple roads receive the same ID. Duplicate intersections were removed so that each road has only one intersection per crossing.

Additionally, sourcesink points overlapping with intersections were removed. This is necessary because such points should function as intersections within the network rather than entry or exit points.

Each intersection is assumed to have a fixed length of 20 meters, based on an average road width.

1.3 Special Cases and Adjustments

1.3.1 Sourcesink of N104

N104 does not directly connect to N1 but is linked via a Z-road, causing its sourcesink to be offset from the main road. This is not an issue for the network and scenario's, but it does create an extra sourcesink point.

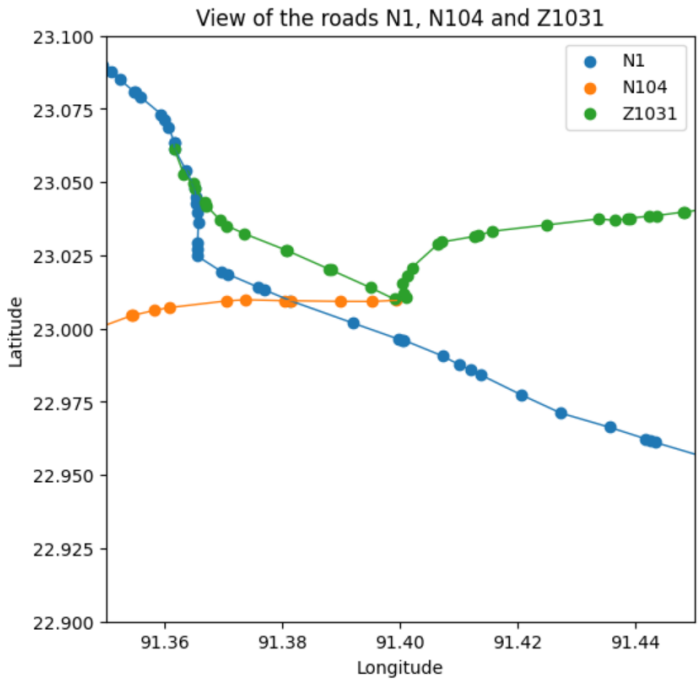

Figure 1: Overview of the N1 road, N104 sideroad and Z1031 road

1.3.2 Disconnection N106

When running the NetworkX code, an error occurred. A brief inspection showed that the issue was located at the connection between the N1 and the N106 side road, as shown in Figure 2.

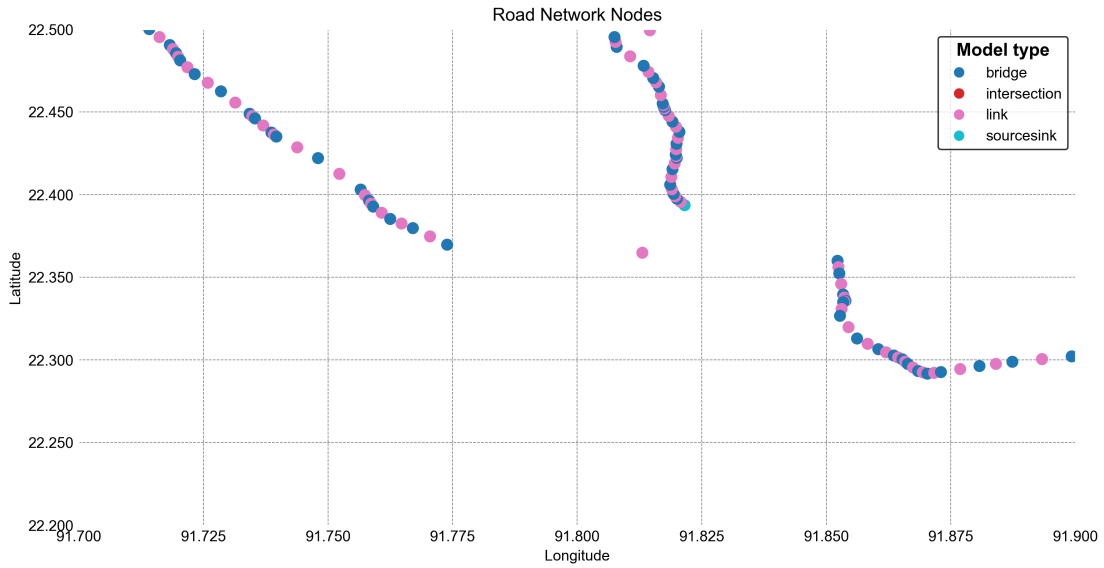

Figure 2: Disconnection of the N1 road and N106 sideroad

Figure 2 shows that the N106 is not properly connected to the N1, as it does not have an intersection with N1. This is likely caused by an inconsistency in the data around Chittagong, where part of the route may be labelled as a Z road instead of an N1 road. As a result, the intersection between the N1 and N106 was not correctly detected.

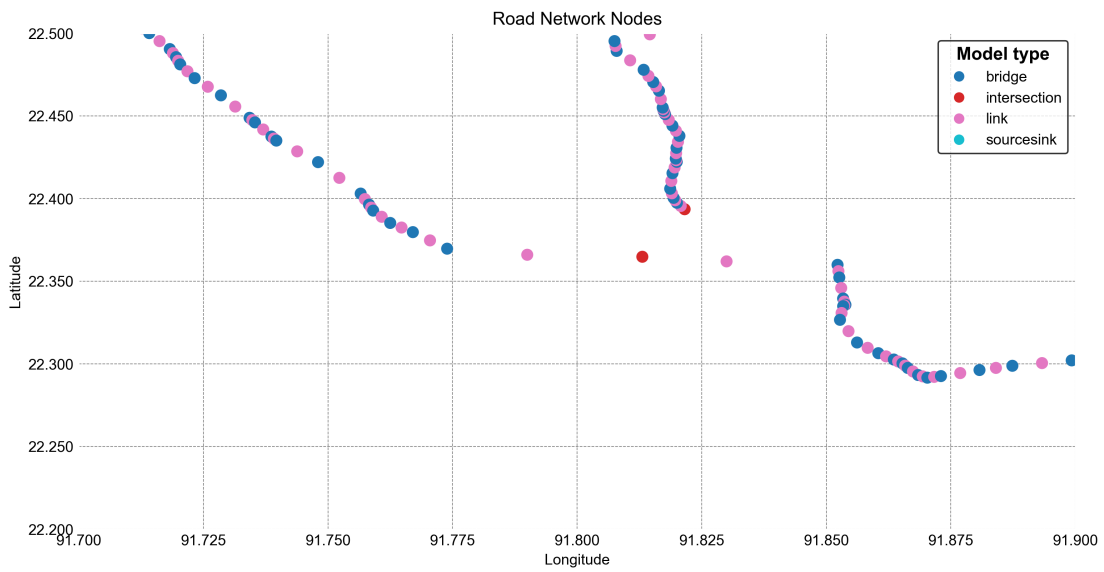

Figure 3: The fixed intersection with links between N1 and N106

To address this issue, an intersection was added between the N1 and N106, see Figure 3. The intersection between the N1 and N106 is shown twice in the image, since we changed the sourcesink of N106 to an intersection, but it is included only once in the NetworkX model because both intersections have the same ID. Furthermore, in the NetworkX model, every bridge, intersection, and sourcesink must be connected by a link.

However, this connection was still missing between the N1 and N106. To fix this, the distance between the two N1 points was calculated, after which two links were manually added to the dataset, also visible in Figure 3. The length of each link was set to half of the calculated distance between the two points of N1.

1.4 Data to Model translation

The processed data was translated into a network consisting of links (road segments), intersections, and bridges. Intersection grouping was used to assign consistent IDs across different roads referring to the same physical location.

Finally, connectivity of the network was verified. After manually connecting N106, a fully connected network was obtained, suitable for simulation.

The assumptions made, such as the intersection threshold of 0.05 and the fixed intersection length of 20 meters, simplify reality but make the model computationally tractable. These assumptions influence the accuracy of the network and should be validated in further analysis.

The manual adjustments needed for the N106 side road highlight a broader challenge in simulation research: the quality of numerical data. As Marsden and Pingry 2018 and Dong 2019 explain, the quality of input data is not just a technical issue. It also affects how reliable and repeatable the simulations results are.

2 Model Design & Routing Mechanisms

2.1 Model adjustments

The same adjustments to the model have been implemented as in Assignment 2:

- Scenario implementation
- Bridges have the ability to break down
- Data collector gathers data on the average driving time per run

In this section the other adjustments to the model will be explained to make it fit-for-purpose to make sure the paths to the vehicle's destination follows the shortest-path algorithm.

2.1.1 Generating a NetworkX graph

In the `model.py` during the initialization of `BangladeshModel` class, an additional line has been added to initialize a NetworkX graph, see Figure 4.

```
class BangladeshModel:
    def __init__(...):
    ...
    ...
    self.G = nx.Graph()

```

Figure 4: Code excerpt of model initialization with the addition of initializing a NetworkX graph

Then, in the `generate_model()` method, the following changes have been implemented:

- input file is added as a variable before the method is called (see Figure 5)
- roads (road names) are extracted from the dataframe over manually inserting them (see Figure 5)
- an empty dictionary (network_dict) is initialized to collect node information and length of edges (links) (see Figure 6)
- after each type of infrastructure is initialized, the data is added to the dictionary
- after the for-loop ends, the `generate_network()` method is called to add the nodes and edges to the graph

```
file_name = '../data/data_intersectionsN1N2.csv'

def generate_model(self):
    df = pd.read_csv(self.file_name)

    roads = list(df.road.unique())
```

Figure 5: Code excerpt of model initialization with the addition of initializing a NetworkX graph

```
network_nodes = "bridges": [], "sourcesinks": [], "intersections": [], "links": {}
```

Figure 6: Code excerpt of the empty network_dict inside the generate_model() method

Then, after each agent (Infrastructure) is initialized and added to the model, the data is added to the network_nodes dictionary. While potentially misleading, during this step the links are also added to the "network_nodes", in order to add the length of the road to the edge later on.

Finally, the method `generate_network()` is called with the network_dict as it's parameter.

In the `generate_network()` method the nodes are added. Then while looping over all the paths from the `path_ids_dict`, each edge is added to a temporary `edge_list`, which stores the pairs of nodes with their given length.

```
        elif model_type == 'sourcesink':
            agent = SourceSink(row['id'], self, row['length'], name, row['road'])
            self.sources.append(agent.unique_id)
            self.sinks.append(agent.unique_id)
            network_nodes["sourcesinks"].append((row['id'], row['length'], row['lon'], row['lat']))
        elif model_type == 'bridge':
            agent = Bridge(row['id'], self, row['length'], name, row['road'], condition=row['condition'], broken_chance=self.scenario[row['condition']])
            network_nodes["bridges"].append((row['id'], row['length'],row['lon'], row['lat']))
        elif model_type == 'link':
            agent = Link(row['id'], self, row['length'], name, row['road'])
            network_nodes["links"][row['id']] = row['length']
        elif model_type == 'intersection':
            if not row['id'] in self.schedule._agents:
                agent = Intersection(row['id'], self, row['length'], name, row['road'])
                network_nodes["intersections"].append((row['id'], row['length'], row['lon'], row['lat']))
    self.generate_network(network_nodes)
```

Figure 7: Code excerpt of adding the node data to the network dictionary inside the generate_model() method and calling the generate_network method

While this also adds edges multiple times to the list, this is no issue for NetworkX as duplicate edges will be treated as a single edge.

```
        def generate_network(self, network_dict):
            #Add bridges as nodes to network
            bridges = network_dict["bridges"]
            bridges_with_length = [(bridge[0], {'weight': bridge[1], 'type': 'bridge', 'lon': bridge[2], 'lat': bridge[3]}) for bridge in bridges]
            self.G.add_nodes_from(bridges_with_length)

            sourcesinks = network_dict["sourcesinks"]
            sourcesinks_nodes = [(sourcesink[0], {'weight': sourcesink[1], 'type': 'SoSi', 'lon': sourcesink[2], 'lat': sourcesink[3]}) for sourcesink in sourcesinks]
            self.G.add_nodes_from(sourcesinks_nodes)

            intersections = network_dict["intersections"]
            intersections_nodes = [(intersection[0], {'weight': intersection[1], 'type': 'intersection', 'lon': intersection[2], 'lat': intersection[3]}) for intersection in intersections]
            self.G.add_nodes_from(intersections_nodes)

            links = network_dict["links"]

            paths = self.path_ids_dict
            edge_list = []
            for path_key in paths:
                path = paths[path_key]
                for i in range(len(path) - 1):
                    if path.iloc[i] in links:
                        edge_list.append((path.iloc[i-1], path.iloc[i+1], links[path.iloc[i]]))
            self.G.add_weighted_edges_from(edge_list)

```

Figure 8: Code excerpt of generate_network() method

With the NetworkX graph implemented in `self.G` in the `BangladeshModel` class, the NetworkX graph and methods can be accessed anywhere the model is initialized.

2.2 Routing and path allocation

To direct a vehicle to the right destination, the path to it's destination is assigned to the vehicle. This is done either on the path_ids_dict if the path from origin to destination exists or the shortest path to the destination is generated based on NetworkX's shortest_path algorithm. This is then added to the path_ids_dict so it can be accessed by other vehicles in subsequent steps.

The implementation of the shortest path inside the `get_route` function can be seen in Figure 9

```
        def get_route(self, source):
            """
                Picks a random sink and generates the shortest path from the specified source to this sink.
                First looks up if a path has been generated before, otherwise generates new one from networkX model.
            """
            while True:
                # different source and sink
                sink = self.random.choice(self.sinks)
                if sink is not source:
                    break
            if (source, sink) in self.path_ids_dict:
                return self.path_ids_dict[source, sink]
            else:
                shortest_path =  nx.shortest_path(self.G, source=source, target=sink, weight='weight')
                self.path_ids_dict[source, sink] = shortest_path
                return shortest_path

```

Figure 9: Code excerpt of get_route() method

Currently vehicles follow routes based on a fixed shortest-path algorithm. This assumes they always have complete information about the network. However Keller and Hu 2019 argue that simulations should instead consider that agents often have incomplete and changing information. Next to that, the shortest path might not always take the shortest time, as bridges might break down and other traffic delays are not fully taken into account.

2.3 Integration between MESA and NetworkX

In figure 10 the integration between MESA and NetworkX is shown, as described in the previous two subsections.

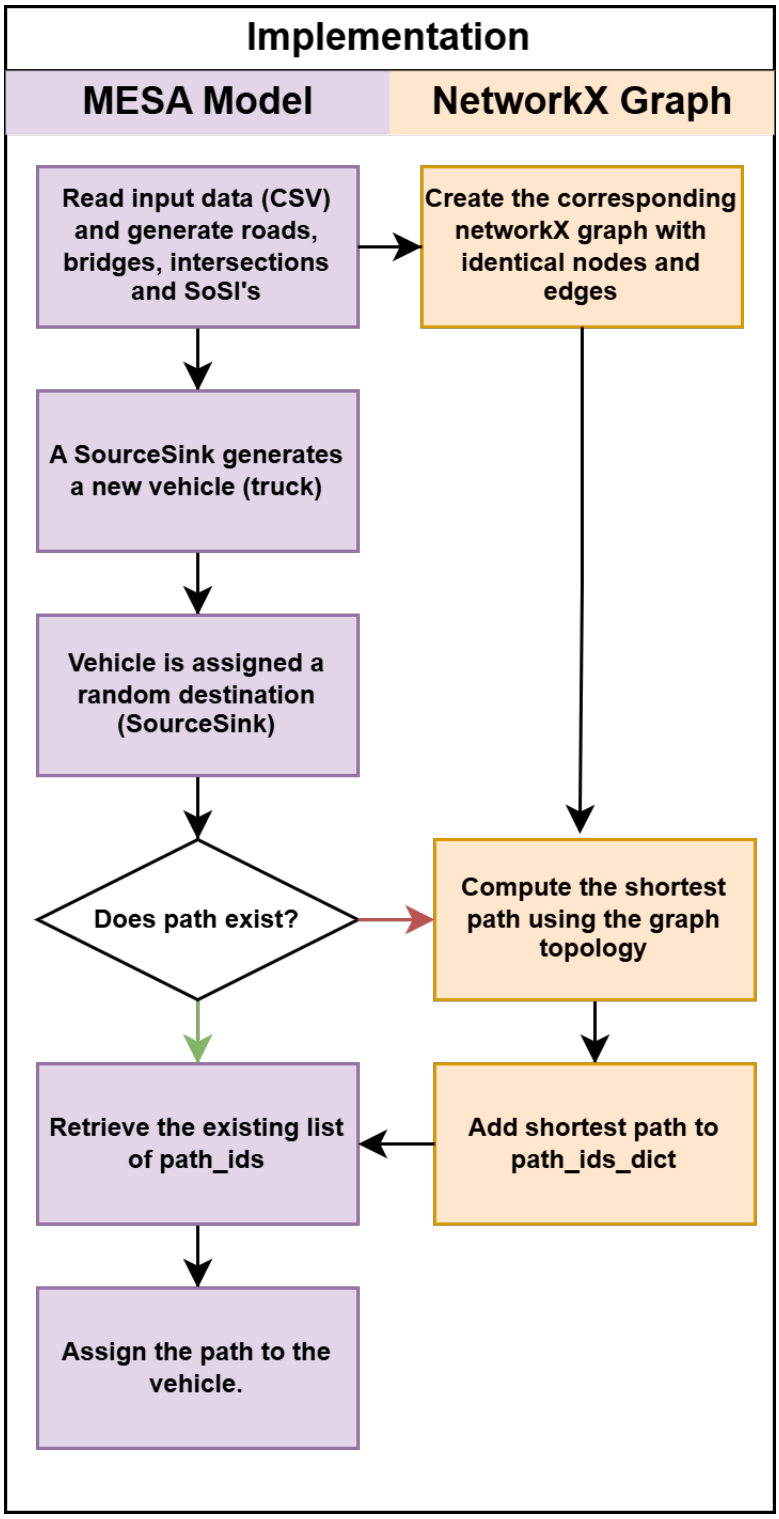

Figure 10: Flowchart of integration between MESA and NetworkX

3 Experiments & Scenarios

For the analysis, ten simulations are run for each scenario, with a different random seed recorded for each simulation to ensure reproducibility and consistency. In each scenario, five days are simulated in minutes (7200 ticks). Using a for loop, all scenarios are calculated and saved to a CSV file. As shown in Table 1, five scenarios were defined, each with a different probability distribution for bridge failure across categories A to D. This makes it possible to examine how differences in failure probability distributions, in combination with the bridge conditions, which remain constant across all scenarios, affect travel time through the network.

Table 1: Probability distribution of bridge failure across categories A to D for each scenario.

Scenario	Cat A (%)	Cat B (%)	Cat C (%)	Cat D (%)
0	0	0	0	0
1	0	0	0	5
2	0	0	5	10
3	0	5	10	20
4	5	10	20	40

During each simulation run, a truck is generated every five minutes and assigned a random route through the network. This route is determined by a randomly selected source and sink node. By generating a large number of trucks (7,200 / 5 = 1,440) and allowing them to traverse random routes, a robust estimate of the average truck driving time in the network can be obtained, which is calculated at the end of the simulation. Combined with the different scenarios, this approach provides insight into how the probability distribution of bridge failures affects network delays and, ultimately, the average driving time of trucks.

The scenarios were run in a clear and organized way, making it easier to see and understand the effects of different factors. This approach avoids a random model design, which can mix up effects and make it hard to see how each factor influences the results Sanchez et al. 2021.

Besides this, the model structure follows systematic decomposition principles. According to Hofmann 2004, using clear criteria to decompose systems into individual components is essential for simulations to remain manageable and valid.

4 Results & Discussion

4.1 Network composition and bridge distribution

To interpret the simulation results, the network composition was analyzed. Figure 11 shows the distribution of bridge categories across N1, N2, and their side roads.

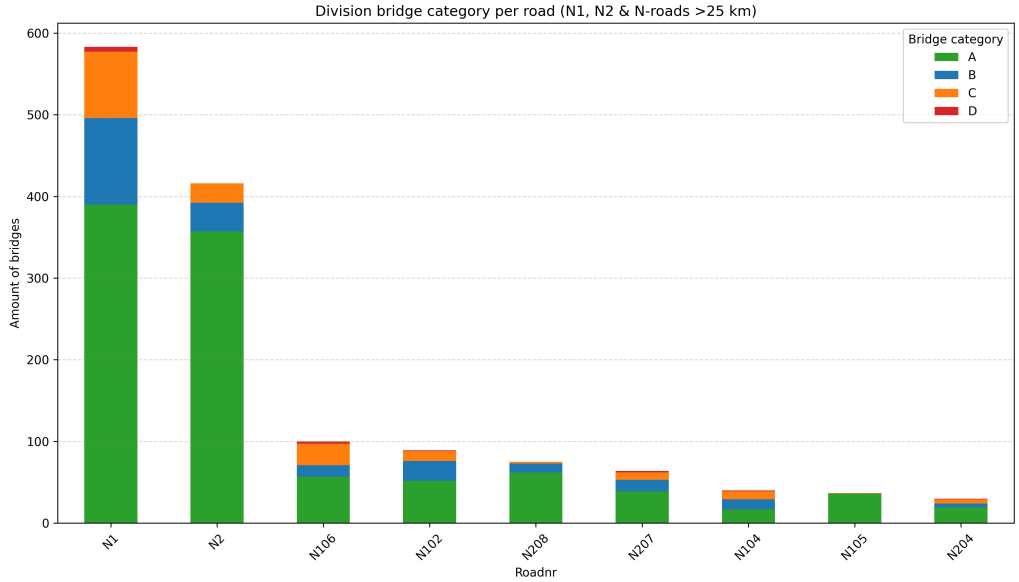

Figure 11: Distribution of bridge categories (A, B, C, and D) per road segment.

As shown in Figure 11, the network is dominated by Category A bridges, particularly on the N1 and N2. In contrast, Category D bridges are rare, with only a few on the N1 and none on the N2. This distribution helps explain the network's resilience to minor disruptions.

4.2 Results of the Scenario

The boxplot in Figure 12 presents the distribution of average driving times across the five scenarios (S0–S4).

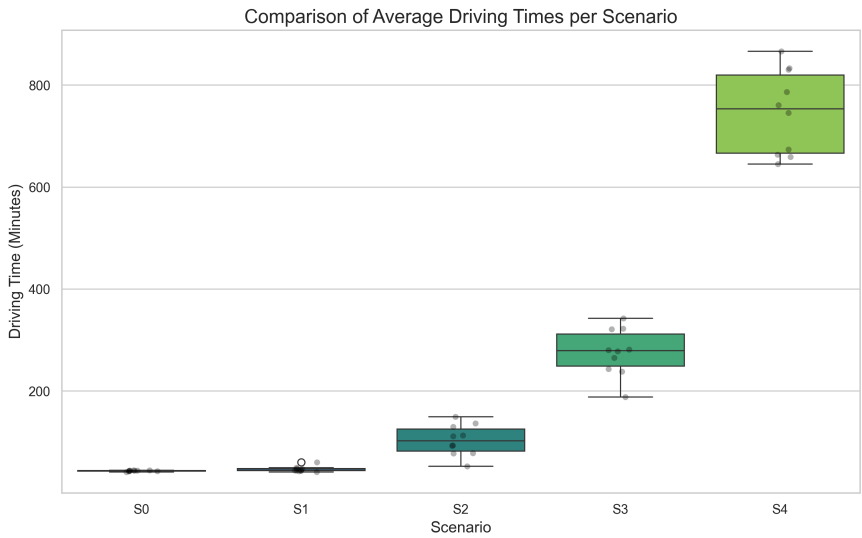

Figure 12: Distribution of driving times across five scenarios on the N1 and N2 network.

In the base case (S0), travel times are stable between 40–45 minutes, as seen in Figure 12. Scenario 1 shows

almost no change, which aligns with Figure 11, where Category D bridges are scarce.

In Scenario 2, Figure 12 shows a clear increase in median travel time to around 100 minutes, indicating growing variability. A sharp increase follows in Scenarios 3 and 4: in S3, median travel time reaches approximately 280 minutes, while in S4 it exceeds 750 minutes.

4.3 Discussion: differences between scenario outcomes

The interconnected structure of the N1/N2 network changes how failures affect the system compared to a simple linear network. This is clearly illustrated in Figure 12.

First, Scenario 1 shows almost no difference from the base case (S0), indicating limited impact of minor disruptions. As shown in Figure 11, Category D bridges are nearly absent on the main N1 and N2, meaning that failures mainly occur on side roads while primary routes remain unaffected.

A clear tipping point occurs between Scenarios 2 and 3. As seen in Figure 12, median travel time increases sharply, which can be explained by the failure of more critical Category B bridges (Figure 11). These bridges are concentrated on key segments of the N1 and N2, and their failure creates bottlenecks at intersections, forcing multiple flows into the same delayed routes.

The extreme increase in Scenario 4 reflects a non-linear accumulation of delays. Given the large number of Category A bridges (over 700, see Figure 11), even a small failure probability significantly disrupts the main network structure. Finally, the wide spread in Scenario 4 (Figure 12) highlights increased unpredictability. Travel times vary depending on whether routes pass through disrupted hubs, demonstrating the sensitivity of the network to failures at key intersections.

Overall, the scenarios reveal a clear progression: minor side-road failures (S1) have little effect, moderate failures (S2) introduce variability, critical bridge failures (S3) create severe bottlenecks, and widespread failures (S4) lead to extreme, unpredictable delays.

4.4 Key relationships between KPIs

The results highlight several important relationships between key performance indicators. First, there is a clear link between connectivity and travel time. Once the frequently occurring Category B bridges (141 in total on the N1/N2, see Figure 11) begin to fail, the average travel time increases sharply (Figure 12). This demonstrates that overall network performance strongly depends on these critical connections.

Second, a trade-off emerges between shortest path and network reliability. While the NetworkX model identifies the shortest path, Figure 11 shows that Category A bridges are the most numerous. As failure rates increase, the shortest path becomes less a measure of efficiency and more a measure of risk, as it is likely to include one of these frequently disrupted bridges. This supports the findings of Jafino et al. 2020, who argue that transport network criticality is dynamic and depends on both infrastructure conditions and the metrics used for evaluation.

Finally, variance can be interpreted as an indicator of system stability. The large spread observed in Scenario 4 (Figure 12) shows that travel times become highly unpredictable. This reflects the sensitivity of the interconnected network to disruptions at key nodes and intersections.

It is important to note that no single KPI fully captures network performance. As Jafino et al. (2020) emphasize, criticality can be assessed through multiple dimensions, none of which is superior. Therefore, the selection of KPIs should be guided by the specific policy context and the priorities of decision-makers.

5 Limitations and possible improvements of the model

The simulation is limited by simplifying assumptions in network generation. For instance, intersections are detected using a 0.05 threshold (50 m) and assigned a fixed length of 20 m, which reduces complexity but may distort the network topology. Data inconsistencies also required manual corrections (e.g., N106 connectivity), and subtracting bridge lengths sometimes produced negative node lengths, resolved by taking absolute values. Lastly, in a small (0.012%) amount of cases the lengths of links and sourcesinks was not calculated correctly due to some links having a very short length and thus going below zero when subtracting the length of bridges. This was solved by taking the absolute value of the length, which means the length of bridges was, in these cases, coutned double. Because it is such a small percentage and the delay times are independent of this, it does not lead to significant changes in results

Model dynamics are also simplified. Routes are based solely on geographic distance and do not adapt to real-time conditions, as edge weights remain static. Disruptions (e.g., bridge closures) therefore do not trigger rerouting. Additionally, trucks can only travel between predefined origin–destination points, limiting realism.

An improvement would be dynamic shortest-path routing, where edge weights update in real time to reflect network conditions. Allowing trucks to travel to any node would further enhance realism, though both changes would require significant modifications to the model structure. This would transform the model in a system where routing reflects real-time network criticality and operational shifts as suggested by Jafino et al. 2020.

References

Dong, John Qi (2019). "Numerical data quality in simulation research: A reflection and epistemic implications". In: *Decision Support Systems* 126. DOI: 10.1016/j.dss.2019.113134.

Hofmann, Marko A. (2004). "Criteria for Decomposing Systems Into Components in Modeling and Simulation: Lessons Learned with Military Simulations". In: *SIMULATION* 80.7-8. DOI: 10.1177/0037549704049876.

Jafino, Bramka Arga, Jan Kwakkel, and Alexander Verbraeck (2020). "Transport network criticality metrics: a comparative analysis and a guideline for selection". In: *Transport Reviews* 40.2, pp. 241–264. DOI: 10.1080/01441647.2019.1703843.

Keller, Nicholas and Xiaolin Hu (2019). "Towards Data-Driven Simulation Modeling for Mobile Agent-Based Systems". In: *ACM Transactions on Modeling and Computer Simulation* 29.1. DOI: 10.1145/3289229.

Marsden, James R. and David E. Pingry (2018). "Numerical data quality in IS research and the implications for replication". In: *Decision Support Systems* 115, A1–A7. ISSN: 0167-9236. DOI: 10.1016/j.dss.2018.10.007.

Sanchez, Susan M., Paul J. Sanchez, and Hong Wan (2021). "Work Smarter, Not Harder: A Tutorial on Designing and Conducting Simulation Experiments". In: *Proceedings of the 2021 Winter Simulation Conference*. Piscataway, NJ: IEEE, pp. 1128–1142.

Acknowledgements

AI Statement

This report was prepared with the assistance of AI tools. All concepts, prompts, and inputs, however, originated from the authors. AI was used solely as a supplementary resource in two specific areas:

- Coding: AI was used to resolve coding issues when the team faced challenges they couldn't address independently. Prompts were formulated precisely, as the team understood the data and analyses but occasionally needed support with implementation details.
- Writing: AI also supported the writing process by enhancing the flow, clarity, and overall structure of the text. The underlying content, analysis, and conclusions remain entirely the work of the authors.

Group Statement

Underneath, the role and contribution of each group member is described. Overall, the assignment was completed as a group effort, and everyone contributed to the final version of the report.

- Niels van den Dool: Set up the Git repository and LaTeX report, and worked on the Mesa and NetworkX code, wrote section 2.
- Eli Kapteijn: Worked on the Mesa and networkX code, and data cleaning and preparation and checking the whole report.
- Nick Schreurs: Worked on the data cleaning, writing the data cleaning and scenario sections, and finalizing the entire report.
- Lieke Wahlen: Worked on the data cleaning and writing the data cleaning.
- Mirte Wildeboer: Worked on the data cleaning, wrote the results and limitations and checking and finalizing whole report.